

A Bit-Vector Linear Correlation Model for Modular Addition by a Constant (BvCorr-FLAT)

Twilight-Dream

Abstract. We study the Walsh correlation of modular addition by a fixed constant,

$$y = x \boxplus c \pmod{2^n},$$

and the corresponding subtraction primitive via two's complement. Starting from the classical carry-state viewpoint for modular addition and borrowing the bit-vector design philosophy that has proved successful on the differential side, we present *BvCorr-FLAT*, a run-aware linear-correlation model for the variable-plus-constant case.

The central observation is that, inside a maximal run of equal bits of the constant, the single-bit carry kernels indexed by $t_i = \alpha_i \oplus \beta_i$ commute. This allows every $\beta = 0$ interval to be collapsed into a closed-form power update. The only genuinely non-commuting events occur at positions where $\beta_i = 1$; these are handled explicitly through a β -sparse partition. This yields an exact flattened evaluator after run decomposition.

We then introduce a *Binary-Lift* that extracts the length-1 carry layer into a purely bitwise parity term and leaves a residual carry contribution to be handled by a windowed flattened evaluator. The resulting estimator admits a deterministic absolute error bound

$$|C(\alpha, \beta) - \widehat{C}_L(\alpha, \beta)| \leq 2 \text{wt}(\beta) 2^{-L},$$

which immediately yields a conservative linear-weight bound. The framework extends to cascades of constant additions under a product-estimator model.

The presentation keeps the algebra, the algorithmic skeleton, and the reference implementation aligned, so that the reader sees not only the matrix recurrences but also the corresponding evaluator structure.

Keywords: No keywords given.

1 Introduction

Modular addition is the main source of nonlinearity in many ARX primitives. For the two-variable operation $x \boxplus y$, the linear-correlation structure is classical: Wallén gave an exact treatment in terms of carry recurrences and efficient algorithms for modular addition and subtraction [Wal03]. Schulte-Geers later

showed that ordinary addition modulo 2^n is CCZ-equivalent to a quadratic vectorial Boolean function and obtained explicit Walsh-transform results for the two-variable case [SG13]. On the differential side, Azimi *et al.* developed a successful bit-vector framework for addition by a constant [ARS⁺20].

This paper focuses on the practically important *one-constant* linear setting

$$y = x \boxplus c.$$

Our goal is not to reproduce the full two-variable theory in a different costume. Instead, we exploit structure that is special to the fixed-constant branch: the carry dynamics split into maximal 0-runs and 1-runs of the constant, and these runs admit a flattened treatment that is especially natural in a bit-vector implementation.

The contributions of this rewritten version are as follows.

- (i) We formalize an *exact* flattened evaluator based on run decomposition, carry-state row vectors, and a β -sparse partition.
- (ii) We state the run-local commutativity and closed-form power identities in a notation consistent with the implementation.
- (iii) We isolate the length-1 carry layer through a *Binary-Lift*, leaving residual masks $(t_{\text{res}}, \beta_{\text{res}})$ for the windowed estimator.
- (iv) We present a deterministic error bound for the windowed estimator and derive a conservative linear-weight estimate.
- (v) We separate what is proved exactly in this paper from what is merely an implementation heuristic or engineering recommendation.

A recurring theme is that the exact and approximate evaluators are built from the same algebraic core. The approximate side is not a different model; it is a restricted application of the exact run-flattened kernel.

2 Preliminaries and notation

We work over $\mathbb{F}^n$ with little-endian indexing: bit 0 is the least significant bit and bit $n - 1$ is the most significant bit. For $x \in [0, 2^n)$ we write

$$x = \sum_{i=0}^{n-1} x_i 2^i, \quad x_i \in \mathbb{F}.$$

Definition 1 (Walsh correlation). Let $\alpha, \beta \in \mathbb{F}^n$ and let $y = x \boxplus c \pmod{2^n}$. The Walsh correlation of the approximation (α, β) is

$$C(\alpha, \beta) = \mathbb{E}_{x \in \mathbb{F}^n} (-1)^{\alpha \cdot x \oplus \beta \cdot y}.$$

Define

$$t = \alpha \oplus \beta.$$

Let $s_0 = 0$ and define the carry recursion

$$s_{i+1} = \text{maj}(x_i, c_i, s_i), \quad y_i = x_i \oplus c_i \oplus s_i.$$

Then

$$C(\alpha, \beta) = (-1)^{\beta \cdot c} \mathbb{E}_x (-1)^{t \cdot x \oplus \sum_{i=0}^{n-1} \beta_i s_i}.$$

Definition 2 (Runs of the constant). Decompose the fixed constant c into maximal intervals of equal bits,

$$\mathcal{R}(c) = \{[a_k, b_k]\}_{k=1}^r,$$

where each interval is either a 0-run or a 1-run. We write $r = \text{runs}(c)$. Inside a 0-run one has

$$s_{i+1} = x_i \wedge s_i,$$

whereas inside a 1-run one has

$$s_{i+1} = x_i \vee s_i.$$

Definition 3 (Left band and working set). For a window length $L \geq 0$, define

$$\text{LeftBand}(\beta, L) = \bigcup_{i: \beta_i=1} ([i-L, i] \cap [0, n-1]).$$

The corresponding working set is

$$S = \text{supp}(t) \cup \text{LeftBand}(\beta, L).$$

Definition 4 (Linear weight). The linear weight is

$$w(\alpha, \beta) = -\log_2 |C(\alpha, \beta)|.$$

For an estimator $\widehat{C}_L$ with deterministic error budget δ , we define the conservative weight

$$\widehat{w}_L = -\log_2 \left(\max\{|\widehat{C}_L| - \delta, 2^{-n}\} \right).$$

3 Exact run-flattened evaluator

3.1 Carry-state kernels

We use a two-state row vector (V_0, V_1) , where the two coordinates correspond to the current carry state $s \in \{0, 1\}$. At bit position i , the update depends on the run type and on t_i .

Lemma 1 (Single-bit kernels). *Inside a 0-run, the single-bit kernels are*

$$K_0 = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix}, \quad K_1 = \begin{bmatrix} 0 & 0 \\ \frac{1}{2} & -\frac{1}{2} \end{bmatrix}.$$

Inside a 1-run, the kernels are

$$K'_0 = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ 0 & 1 \end{bmatrix}, \quad K'_1 = \begin{bmatrix} \frac{1}{2} & -\frac{1}{2} \\ 0 & 0 \end{bmatrix}.$$

Proof. Condition on $x_i \in \{0, 1\}$ and on the current carry state $s_i \in \{0, 1\}$. The phase contribution is

$$(-1)^{t_i x_i \oplus \beta_i s_i},$$

and the next carry is given by the AND-chain or OR-chain recurrence according to the run type. Averaging over x_i gives the four matrix entries directly. $\square$

Lemma 2 (Run-local commutativity). *Inside a fixed run,*

$$[K_0, K_1] = 0, \quad [K'_0, K'_1] = 0.$$

Proof. Direct multiplication gives

$$K_0 K_1 = K_1 K_0 = \begin{bmatrix} 0 & 0 \\ \frac{1}{2} & -\frac{1}{2} \end{bmatrix},$$

and similarly for the primed pair. $\square$

Lemma 3 (Closed forms for powers). *For every integer $k \geq 1$ one has*

$$K_0^k = \begin{bmatrix} 1 & 0 \\ 1 - 2^{-k} & 2^{-k} \end{bmatrix}, \quad K_1^k = \begin{bmatrix} 0 & 0 \\ (-1)^{k-1} 2^{-k} & (-1)^k 2^{-k} \end{bmatrix},$$

and

$$(K'_0)^k = \begin{bmatrix} 2^{-k} & 1 - 2^{-k} \\ 0 & 1 \end{bmatrix}, \quad (K'_1)^k = \begin{bmatrix} 2^{-k} & -2^{-k} \\ 0 & 0 \end{bmatrix}.$$

Proof. A straightforward induction on k suffices. The commuting property lets one permute same-run kernel products when collapsing a $\beta = 0$ interval into a power update. $\square$

3.2 The β -sparse partition

Definition 5 (β -sparse partition). Fix a run interval $[i, j]$. Split the relative run positions into alternating $\beta = 0$ intervals and $\beta = 1$ singletons. At a $\beta = 1$ position, the sign-flip matrix

$$D = \text{diag}(1, -1)$$

must be applied in place before the corresponding one-step kernel update.

The point is brutally simple: commutativity holds inside a same-run block only as long as no β -flip interrupts the chain. The β -ones are therefore the synchronization points of the exact evaluator.

Theorem 1 (Exact flattened evaluation). *Assume that the run decomposition of c is given. Then the correlation $C(\alpha, \beta)$ can be computed exactly by:*

- (1) *collapsing every $\beta = 0$ interval inside a run into closed-form powers of K_0, K_1 or K'_0, K'_1 , according to the run type and the counts of t -bits; and*
- (2) *applying D followed by the appropriate one-step kernel at every $\beta = 1$ singleton.*

The exact evaluator returns

$$C(\alpha, \beta) = (-1)^{\beta \cdot c} \frac{V_0 + V_1}{2^n}.$$

If the run decomposition is precomputed, the algebraic update cost is

$$O(\text{runs}(c) + \text{wt}(\beta)).$$

A direct one-pass extraction of runs costs $O(n)$, which is constant in the fixed-word implementation used for $n \leq 64$.

Proof. Inside a run, every maximal $\beta = 0$ interval can be reordered by the commutativity lemma and therefore collapsed into the corresponding closed-form powers. At a $\beta = 1$ position the sign-flip D does not commute with the carry kernel and must be applied exactly at that location. Concatenating all segments reconstructs the original per-bit carry-state chain, hence the result is exact. Because every exact evaluation processes all n bit positions, the accumulated dyadic denominator is 2^n . The stated update cost counts one algebraic update per run interval and one mandatory synchronization event per β -one. $\square$

Remark 1 (Alignment with the implementation). The original markdown draft mixed several mathematically equivalent-looking but implementation-inconsistent update formulas. In particular, the closed-form updates for repeated K_0 , repeated K'_0 , and the two $\beta = 1$ singleton steps must be aligned with the actual row-vector implementation; otherwise the paper silently describes a different algorithm than the code. This rewritten version follows the implementation-level semantics throughout.

4 Binary-Lift

The carry term $\sum_i \beta_i s_i$ contains a particularly simple length-1 contribution, namely the part originating from $x_{i-1} \wedge c_{i-1}$. This piece can be extracted exactly into a parity term.

Lemma 4 (Parity identity for the length-1 carry layer). *For every mask β ,*

$$\bigoplus_{i=0}^{n-1} \beta_i (x_{i-1} \wedge c_{i-1}) = \text{parity}(x \wedge (c \wedge (\beta \ll 1))).$$

Proof. The i -th summand is active exactly when $\beta_i = 1$, $c_{i-1} = 1$, and $x_{i-1} = 1$. That is precisely the parity of the masked vector $x \wedge (c \wedge (\beta \ll 1))$. $\square$

Definition 6 (Binary-Lift masks). Define

$$\text{carry}_1 = c \wedge (\beta \ll 1),$$

and then

$$u = \beta \oplus \text{carry}_1, \quad v = \beta, \quad w = \beta.$$

The residual masks are

$$\beta_{\text{res}} = \beta \wedge \neg \text{carry}_1, \quad t_{\text{res}} = (\alpha \oplus \beta) \oplus \text{carry}_1.$$

Remark 2 (What Binary-Lift does and does not do). Binary-Lift is an exact algebraic extraction of the length-1 carry layer. It is *not* a standalone correlation estimator. Its job is to move a simple carry contribution into a linear parity term and thereby reduce the residual mask weight that is seen by the windowed evaluator.

5 Windowed estimator and certified error bound

5.1 Restricted exact evaluation on the working set

The windowed estimator $\hat{C}_L$ is obtained by running the same exact flattened machinery only on the working set

$$S = \text{supp}(t) \cup \text{LeftBand}(\beta, L),$$

while ignoring carry influence that originates outside S .

Definition 7 (Windowed estimator). Let $\hat{C}_L(\alpha, \beta)$ be the estimator obtained by applying the exact run-flattened evaluator only on the active set S . Bits outside S are ignored as possible sources of carry influence.

In the fixed-word implementation for $n \leq 64$, the left band can be assembled exactly by

$$\text{LeftBand}(\beta, L) = \bigvee_{d=0}^{\min(L, n-1)} (\beta \gg d),$$

which costs $O(\min\{L, n\})$ bitwise word operations.

5.2 Propagation gate bound

Lemma 5 (Per-position propagation bound). *Fix an index i with $\beta_i = 1$. If the carry influencing s_i originates to the left of the window $[i - L, i]$, then along the path from that origin to i there exists a monotone constraint of length at least L on the input bits: an all-ones constraint across a 0-run segment, or dually an all-zeros constraint that prevents recreation inside a 1-run segment. Consequently,*

$$\Pr[\text{missed influence at } i] \leq 2^{-L}.$$

Proof. Inside a 0-run, propagation from left to right requires the carry to survive through

$$s_{j+1} = x_j \wedge s_j,$$

hence every traversed bit on that segment must satisfy $x_j = 1$. Inside a 1-run, once $s_j = 1$ the recurrence

$$s_{j+1} = x_j \vee s_j$$

preserves the carry automatically, but recreation inside the window is possible unless an appropriate all-zeros obstruction is maintained. Therefore any influence that enters from the left and remains invisible to the truncated model must cross a monotone length- L event, whose probability is at most 2^{-L} . $\square$

Theorem 2 (Deterministic absolute error bound). *For every (α, β) and every $L \geq 0$,*

$$|C(\alpha, \beta) - \widehat{C}_L(\alpha, \beta)| \leq 2 \text{wt}(\beta) 2^{-L}.$$

Proof. For each index i with $\beta_i = 1$, let E_i be the event that the truncation misses a carry influence at that position. On E_i the exponent of the $\{\pm 1\}$ integrand flips, so

$$|f - g| \leq 2 \mathbf{1}_{E_i}.$$

Taking expectations gives

$$|\mathbb{E}f - \mathbb{E}g| \leq \mathbb{E}|f - g| \leq 2 \Pr[E_i].$$

By the previous lemma, $\Pr[E_i] \leq 2^{-L}$. A union bound over all β -ones yields the claim. $\square$

Corollary 1 (Conservative weight). *Let*

$$\delta_L = 2 \text{wt}(\beta) 2^{-L}.$$

Then

$$\widehat{w}_L = -\log_2 \left(\max\{|\widehat{C}_L| - \delta_L, 2^{-n}\} \right)$$

satisfies

$$\widehat{w}_L \geq w(\alpha, \beta).$$

Algorithm 1 Offline/online exact run-flattened evaluator

```

Input: alpha, beta, fixed constant c, precomputed run table R(c)
t <- alpha xor beta
V <- (1,0)
for each run rho in R(c) do
  partition rho into maximal beta=0 intervals and beta=1 singletons
  for each beta=0 interval I inside rho do
    count zeros/ones of t on I
    collapse I by the corresponding closed-form power update
  end for
  for each beta=1 singleton j inside rho do
    apply D = diag(1,-1) in place
    apply the one-step carry kernel selected by run type and t_j
  end for
end for
return signed_sum(V, beta, c) / 2^n

```

If a target absolute bias error $\varepsilon > 0$ is prescribed, it is sufficient to choose

$$L = \left\lceil \log_2 \frac{2 \text{wt}(\beta)}{\varepsilon} \right\rceil.$$

With *Binary-Lift*, replace $\text{wt}(\beta)$ by $\text{wt}(\beta_{\text{res}})$ and enforce $L \geq 2$.

Remark 3 (Complexity of the estimator). With straightforward bitwise assembly of the left band, the cost is

$$O(\min\{L, n\} + \text{runs}(c) + \text{wt}(\beta \cap S)).$$

For the fixed-word implementation with $n \leq 64$, this is $O(1)$ in practice. If run decomposition is cached and the left-band preprocessing is implemented by a word-parallel primitive, the visible algebraic core is the same as in the exact evaluator, but restricted to the working set.

6 Algorithmic skeleton and complexity summary

Theorems are easier to audit when the evaluator is also written as an explicit procedure. Algorithms 1 and 2 summarize the exact and restricted kernels in the same offline/online order used by the reference header. The point is not to introduce a different algorithmic layer, but to make the already-proved recurrence look like something one can actually implement, profile, and compare.

A short code-faithful excerpt is also useful, because the implementation relies on an exact doubling construction rather than a naive shift-OR loop:

Listing 1: Doubling-style construction of $\text{LeftBand}(\beta, L)$ used by the reference header.

Algorithm 2 Windowed evaluator with active-run dispatch

```

Input: alpha, beta, fixed constant c, precomputed run table R(c), window
      L
t <- alpha xor beta
S <- supp(t) union LeftBand(beta, L)
A <- active runs of c intersecting S
V <- (1,0)
for each active run rho in A do
  if rho is fully active then
    apply the exact span update on rho
  else if beta is zero on rho then
    collapse rho by counts only
  else if rho is sparse in active events then
    process rho by the sparse bit-event kernel
  else
    process rho by the segmented fallback kernel
  end if
end for
renormalize to denominator 2^n and return the signed dyadic value

```

```

static inline std::uint64_t left_band_u64( std::uint64_t beta, int L, int
      n ) noexcept
{
  const std::uint64_t m = mask_n_u64( n );
  beta &= m;
  if ( L <= 0 ) return beta;

  int remaining = std::min( L, n - 1 );
  std::uint64_t band = beta;
  int covered = 1;
  while ( remaining > 0 )
  {
    const int step = std::min( covered, remaining );
    band |= ( band >> step );
    covered += step;
    remaining -= step;
  }
  return band & m;
}

```

Table 1 should be read together with the estimator remark rather than in isolation. The asymptotic story is intentionally conservative: the implementation-specific dispatch only improves constant factors, whereas the offline/online split and the active-set reduction are the structural statements that belong in the paper.

Table 1: Complexity summary under the offline/online split for a fixed constant c . Here $S = \text{supp}(t) \cup \text{LeftBand}(\beta, L)$ is the working set of the windowed evaluator.

Component	Cost	Role
Run decomposition of c	$O(n)$ offline	Build and cache the fixed-constant run table.
Exact online evaluator	$O(\text{runs}(c) + \text{wt}(\beta))$	One run-level update per run, plus one mandatory synchronization event per β -one.
Left-band assembly	$O(\log \min\{L, n\})$ on the word-RAM used here	Exact doubling-style construction of $\text{LeftBand}(\beta, L)$.
Windowed online evaluator	$O(\log \min\{L, n\} + \text{runs}(c _S) + \text{wt}(\beta \cap S))$	Restricted exact kernel driven only by active runs.
Coarse engineering bound	$O(\log n + \text{runs}(c) + \text{wt}(\beta))$	Convenient upper bound used for quick complexity comparisons.
Binary-Lift mask extraction	$O(1)$ for fixed machine words	Algebraic preprocessing of the length-1 carry layer before windowing.

7 Cascades of constants

Suppose that a product model is used for a sequence of constant additions. For round j , let the true round quantity be $C^{(j)}$ and the estimator be $\hat{C}^{(j)}$ with

$$|C^{(j)} - \hat{C}^{(j)}| \leq \delta_j, \quad |C^{(j)}| \leq 1, \quad |\hat{C}^{(j)}| \leq 1.$$

Proposition 1 (Product estimator with additive error accounting). *Let*

$$C_{\text{tot}} = \prod_{j=1}^m C^{(j)}, \quad \hat{C}_{\text{tot}} = \prod_{j=1}^m \hat{C}^{(j)}.$$

Then

$$|C_{\text{tot}} - \hat{C}_{\text{tot}}| \leq \sum_{j=1}^m \delta_j.$$

Proof. Use the standard telescoping expansion

$$\prod_{j=1}^m a_j - \prod_{j=1}^m b_j = \sum_{j=1}^m \left(\prod_{k < j} a_k \right) (a_j - b_j) \left(\prod_{k > j} b_k \right),$$

then take absolute values and apply the fact that every factor has magnitude at most 1. $\square$

Corollary 2 (Conservative total weight). *Let $\Delta_{\text{tot}} = \sum_j \delta_j$. Then*

$$\hat{w}_{\text{tot}} = -\log_2 \left(\max\{|\hat{C}_{\text{tot}}| - \Delta_{\text{tot}}, 2^{-n}\} \right)$$

is conservative for the product model.

8 Implementation notes

The reference implementation works on $n \leq 64$ bit words and stores the carry-state accumulator in signed wide integers. The core invariants are:

- (1) The row vector (V_0, V_1) is represented exactly as a dyadic rational with denominator 2^{scale} .
- (2) Every update is performed by integer additions, subtractions, and shifts only.
- (3) Floating-point conversion is deferred to the very end.
- (4) For the restricted evaluator, the numerator is renormalized so that the final denominator is exactly 2^n .

These implementation choices matter. Several details must not drift apart between the mathematics and the reference header. In particular, the following items must stay synchronized:

- the closed-form power updates for repeated K_0 and repeated K'_0 ,
- the two $\beta = 1$ singleton updates after the in-place sign flip,
- the exact-vs-windowed denominator handling, and
- the residual-mask definitions produced by Binary-Lift.

If any of those four pieces is written loosely, the paper starts describing an algorithm-shaped mirage. That goblin has now been evicted.

9 Limitations and scope

This paper deliberately stays in the one-constant branch

$$y = x \boxplus c.$$

We do not claim a replacement for the full two-variable theory of modular addition. Likewise, the windowed estimator is an approximation scheme with a certified absolute error bound; it is not a statement about the exact global Walsh structure of the primitive.

Two further caveats are worth stating plainly.

- (i) Dense β masks and very small windows may force the estimator into a regime where the conservative bound dominates the measured magnitude, at which point exact evaluation is preferable.
- (ii) The original draft contained an “approximation-optimality” theorem in an engineering sense. In this rewritten version that claim is intentionally demoted to discussion: a genuine lower-bound theorem would require a more careful computational model and proof than is presently provided.

10 Conclusion

We presented BvCorr-FLAT, a bit-vector linear-correlation model for modular addition by a constant, with a deliberately unified mathematical, algorithmic, and implementation-level presentation. The exact side is a run-aware flattened evaluator based on carry-state kernels, run-local commutativity, and a β -sparse partition. The approximate side is a restriction of the same evaluator to a working set, optionally preceded by a Binary-Lift that extracts the length-1 carry layer. The estimator comes with a deterministic error bound and therefore with a conservative linear-weight estimate.

The main practical message is pleasantly unromantic: once the fixed-constant structure is made explicit, the supposedly fiddly carry chain becomes much less mystical. Most of the algebra collapses inside runs, and the remaining difficulty is concentrated exactly where β says it cannot be ignored.

References

- [ARS⁺20] S. A. Azimi, A. Ranea, M. Salmasizadeh, J. Mohajeri, M. R. Aref, and V. Rijmen. A bit-vector differential model for the modular addition by a constant. In *ASIACRYPT 2020*, volume 12491 of *Lecture Notes in Computer Science*. Springer, 2020.
- [SG13] Ernst Schulte-Geers. On ccz-equivalence of addition mod $2n$. *Designs, Codes and Cryptography*, 66(1):111–127, Jan 2013.
- [Wal03] Johan Wallén. Linear approximations of addition modulo $2n$. In Thomas Johansson, editor, *Fast Software Encryption*, pages 261–273, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.